

Documentație proiect
Sistem de încărcare cu benzi (M23)

Disciplină: Proiect Sincretic 2

Echipa: Cucea Ioan, Covaciu Sebastian-Adelin, Goron Emanuel

Cuprins

Cuprins	2
1. Introducere	4
1.1 Scopul proiectului	4
1.2 Obiective	4
1.3 Tehnologii utilizate	4
2. Analiza problemei	4
2.1 Descrierea procesului fizic	4
2.2 Comportamentul clapetei	5
2.3 Cerințe funcționale	5
Descrierea funcțională 1 (DF1) — comportament de bază	5
Descrierea funcțională 2 (DF2) — extindere	5
Detectarea defectelor	5
2.4 Condiții de funcționare sintetizate	6
2.5 Comportament UX	6
3. Modelarea sistemului	7
3.1 Abordare — mașini de stare finite (FSM)	7
3.2 FSM-ul sistemului global	7
3.3 FSM-ul clapetei	7
3.4 FSM-ul benzii de intrare (M1, M2 — identice)	8
3.5 FSM-ul benzii de ieșire (M3, M4 — identice)	8
3.6 Tabelul de condiții ca motor de reguli	8
4. Arhitectura soluției	9
5. Protocolul de comunicare	9
5.1 Comunicare Simulator ↔ Controller (TCP)	9
Evenimente (Simulator → Controller)	10
Comenzi (Controller → Simulator)	10
Sincronizare (bidirecțional)	10
5.2 Comunicare Controller ↔ Aplicație web (WebSocket)	11
Tipuri de mesaje (Controller → Aplicație)	11
Comenzi (Aplicație → Controller)	11
5.3 Sincronizarea stării la conectare	11
6. Implementare — Simulator	11
6.1 FSM-urile	11
6.2 Evaluatorul de condiții	12
6.3 Orchestratorul procesului	12
6.4 Serverul TCP	12
6.5 Task-ul de perturbații	12
7. Implementare — Controller	13
7.1 Clientul simulatorului (SimulatorClient)	13
7.2 Starea procesului în memorie (ProcessState)	13
7.3 Hub-ul WebSocket (WebSocketHub + WebSocketMiddleware)	13
7.4 Logger și rapoarte	13

7.5 Baza de date	14
8. Implementare — Aplicație de monitorizare	14
8.1 Tehnologii și structură	14
8.2 Store-ul de conexiune	14
8.3 Vizualizarea procesului	14
8.4 Pagini	15
8.5 Responsivitate	15
9. Testare	15
9.1 Teste unitare automate	15
10. Cerințe de sistem	16

1. Introducere

1.1 Scopul proiectului

Proiectul constă în dezvoltarea unei soluții software complete pentru simularea, monitorizarea și controlul unui proces industrial cu evenimente discrete — sistemul de încărcare cu benzi transportoare M23. Soluția include un simulator al procesului, o aplicație de control și monitorizare accesibilă prin rețea, un modul de jurnalizare a evenimentelor și o interfață web pentru operatori.

1.2 Obiective

- Implementarea unui simulator fidel al procesului M23, cu toate stările și tranzițiile definite
- Dezvoltarea unui sistem de control ierarhizat, cu separare clară între nivelul de proces și nivelul de supervizare
- Monitorizarea procesului în timp real prin intermediul unei aplicații web
- Jurnalizarea evenimentelor și generarea de rapoarte de audit
- Asigurarea toleranței la modificări printr-o arhitectură modulară și testabilă

1.3 Tehnologii utilizate

Componentă	Tehnologie
Simulator	C# / .NET 8, console app
Controller	ASP.NET Core 8
Bază de date	Microsoft SQL Server 2022
ORM	Entity Framework Core 8
Aplicație web	SvelteKit, TypeScript, Tailwind CSS
Comunicare proces	TCP (JSON)
Comunicare web	WebSocket
Testare	xUnit

2. Analiza problemei

2.1 Descrierea procesului fizic

Sistemul M23 este un sistem de încărcare cu benzi transportoare compus din:

- **2 benzi de intrare** (M1, M2) — aduc material în sistem
- **2 benzi de ieșire** (M3, M4) — transportă materialul în afara sistemului
- **Un punct de intersecție** cu o clapetă de selecție cu 3 poziții, care determină traseul materialului
- **3 senzori** (S6, S7, S8) — detectează poziția clapetei
- **2 butoane de oprire** (S0 — oprire totală, S5 — oprire benzi de intrare)

Topologia fizică a sistemului este următoarea: M1 și M3 sunt poziționate pe aceeași parte (stânga), M2 și M4 pe cealaltă parte (dreapta). Clapeta de la punctul de intersecție acționează ca un deflector care direcționează materialul.

2.2 Comportamentul clapetei

Clapeta este un element fizic care poate acoperi una dintre căi la punctul de intersecție:

Poziție senzor	Efect fizic	Ruta materialului
S6 (stânga)	Blochează calea spre M3	M1 → M4, M2 → M4
S7 (mijloc)	Nicio deviere, cale dreaptă	M1 → M3, M2 → M4
S8 (dreapta)	Blochează calea spre M4	M1 → M3, M2 → M3

2.3 Cerințe funcționale

Descrierea funcțională 1 (DF1) — comportament de bază

- Doar o singură bandă de intrare poate funcționa la un moment dat
- Banda de intrare funcționează **doar dacă** banda de ieșire corespunzătoare poziției clapetei este pornită
- Operatorul pornește și oprește benzile individual

Descrierea funcțională 2 (DF2) — extindere

DF2 nu reprezintă un mod separat de funcționare, ci o extindere a comportamentului de bază. Diferența apare exclusiv când clapeta este în poziția S7:

- Pe S7, ambele benzi de intrare pot funcționa **simultan** dacă ambele benzi de ieșire (M3 și M4) sunt pornite
- Dacă activarea simultană nu este posibilă (lipsește o bandă de ieșire), se aplică comportamentul DF1

Detectarea defectelor

Senzorii S6, S7, S8 monitorizează poziția clapetei. Deoarece clapeta poate ocupa o singură poziție la un moment dat, activarea simultană a doi senzori indică o eroare hardware (senzori blocați de impurități). În acest caz:

- Toate benzile se opresc imediat
- O alarmă sună timp de 5 secunde

- Sistemul trece în starea FAULT și **rămâne** în această stare până la intervenția operatorului

2.4 Condiții de funcționare sintetizate

Combinând DF1 și DF2, condițiile complete de funcționare ale benzilor de intrare sunt:

Poziție clapetă	Bandă	Condiția de rulare	Condiția de oprire
S6	M1	M4 pornită, M2 oprită	M4 oprită / M2 pornită
S6	M2	M4 pornită, M1 oprită	M4 oprită / M1 pornită
S7	M1	M3 pornită	M3 oprită
S7	M2	M4 pornită	M4 oprită
S7	M1+M2	M3 pornită ȘI M4 pornită	M3 oprită SAU M4 oprită
S8	M1	M3 pornită, M2 oprită	M3 oprită / M2 pornită
S8	M2	M3 pornită, M1 oprită	M3 oprită / M1 pornită

Această tabelă constituie **motorul de reguli** al sistemului și este evaluată la fiecare eveniment nou.

2.5 Comportament UX

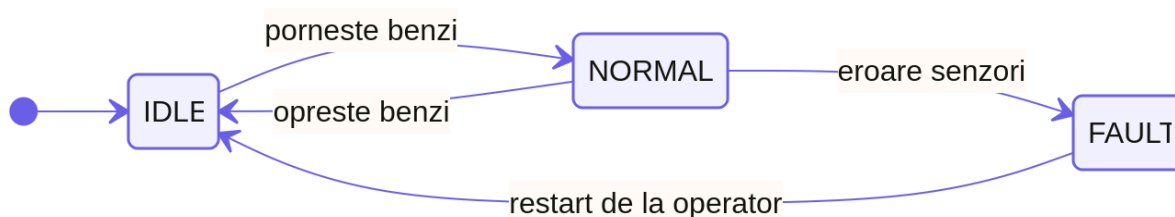
- La pornire: toate benzile oprite, clapeta pe poziția neutră S7
- Benzile de intrare pornite fără condiție îndeplinită intră în starea READY (vizualizate distinct)
- Dacă condiția se îndeplinește ulterior, benzile trec automat din READY în RUNNING
- Dacă condiția se pierde în timp ce o bandă rulează, banda revine în READY (nu în STOPPED), păstrând intenția operatorului
- S0 și S5 trec benzile direct în STOPPED, ignorând starea READY

3. Modelarea sistemului

3.1 Abordare — mașini de stare finite (FSM)

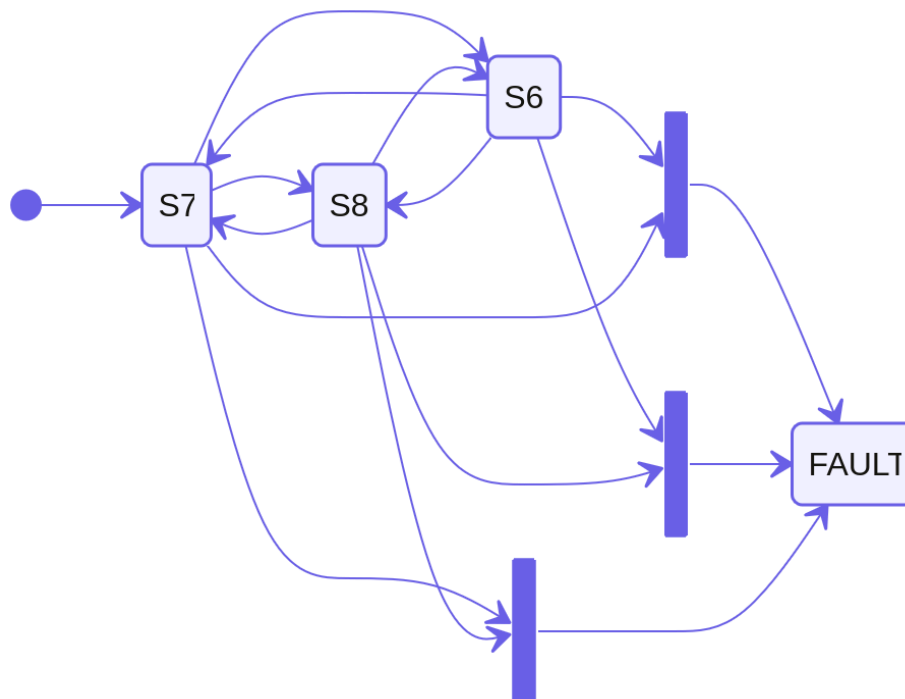
Sistemul este modelat ca un ansamblu de mașini de stare finite (FSM) care rulează în paralel și se influențează reciproc prin evenimente și condiții. Fiecare componentă fizică are propriul FSM, iar un orchestrator central coordonează interacțiunile.

3.2 FSM-ul sistemului global



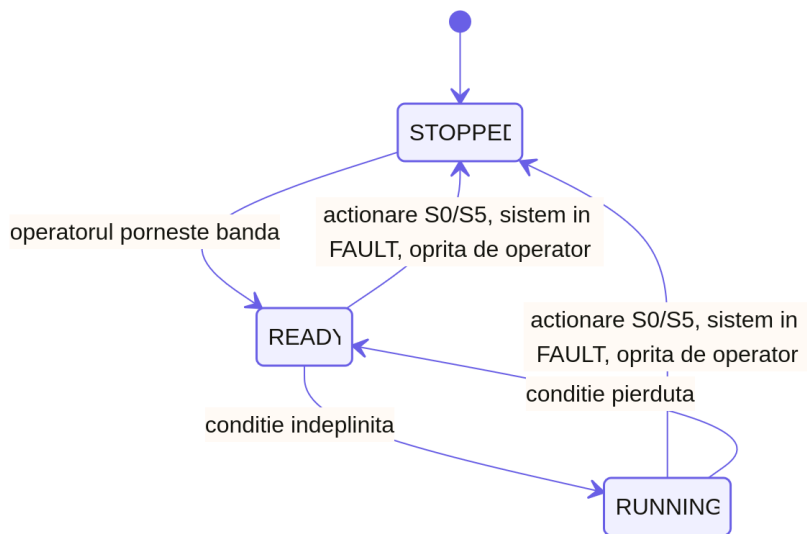
Decizie de proiectare: Starea FAULT este accesibilă doar din NORMAL, nu din IDLE. Motivul: un sistem inactiv nu are consecințe operaționale dacă un senzor este defect; monitorizarea defectelor are sens doar când procesul este activ.

3.3 FSM-ul clapetei



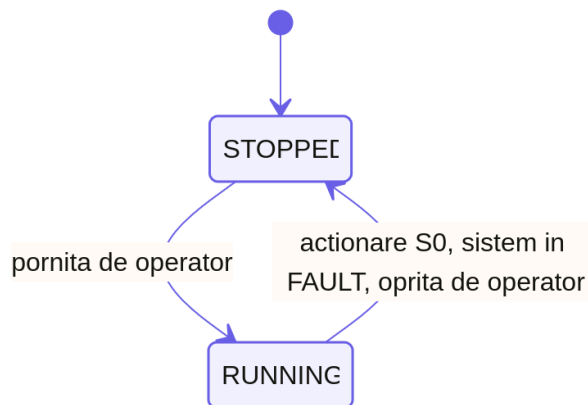
Tranziția spre FAULT necesită **două condiții simultane** — un exemplu de condiție compusă care nu poate fi exprimată simplu într-un FSM clasic, motiv pentru care este modelată explicit prin evenimente concurente.

3.4 FSM-ul benzii de intrare (M1, M2 — identice)



Distincție cheie: READY vs STOPPED. READY înseamnă că operatorul vrea ca banda să meargă, dar condițiile nu permit încă. STOPPED înseamnă că operatorul a oprit-o explicit.

3.5 FSM-ul benzii de ieșire (M3, M4 — identice)



Benzile de ieșire nu au stare READY — ele pornesc imediat la comanda operatorului, fără condiții.

3.6 Tabelul de condiții ca motor de reguli

La fiecare eveniment din sistem (pornire/oprire bandă, schimbare poziție clapetă), orchestratorul reevaluează toate benzile de intrare active (READY sau RUNNING) folosind tabelul de condiții din secțiunea 2.4:

eveniment apare

→ reevaluează M1 și M2

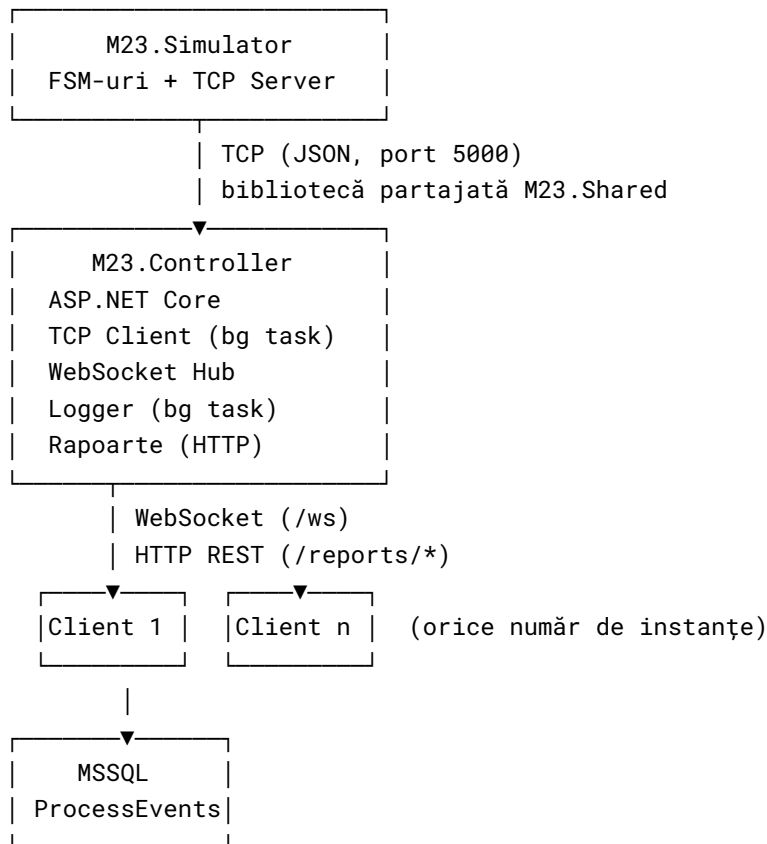
→ dacă bandă RUNNING și condiție pierdută → trece în READY

→ dacă bandă READY și condiție îndeplinită → trece în RUNNING

→ dacă bandă STOPPED → ignoră (operatorul nu a cerut pornirea)

4. Arhitectura soluției

Soluția este împărțită în trei procese distincte care comunică prin rețea:



Motivația separării:

- Simulatorul poate rula independent, testabil fără interfață
- Controller-ul poate fi restartat fără a afecta simulatorul
- Mai mulți clienți web se pot conecta simultan la același controller
- Separarea respectă cerința de "comunicare într-un fir de execuție sau proces separat"

5. Protocolul de comunicare

5.1 Comunicare Simulator ↔ Controller (TCP)

Mesajele sunt transmise ca JSON peste o conexiune TCP persistentă. Formatul general:

```
{  
  "type": "event | command | sync",  
  "timestamp": "2026-06-17T10:23:11.000Z",  
  "payload": { ... }  
}
```

Evenimente (Simulator → Controller)

```
{
  "type": "event",
  "timestamp": "2026-06-17T10:23:11.000Z",
  "payload": {
    "source": "M1 | M2 | M3 | M4 | FLAP | SYSTEM | ALARM",
    "from": "starea anterioară",
    "to": "starea nouă"
  }
}
```

Câmpurile **from** și **to** conțin valorile stărilor din FSM-ul sursei respective. Sunt folosite pentru jurnalizare — controllerul are nevoie de ambele stări pentru a construi un jurnal de audit complet.

Comenzi (Controller → Simulator)

```
{
  "type": "command",
  "timestamp": "2026-06-17T10:23:11.000Z",
  "payload": {
    "target": "M1 | M2 | M3 | M4 | FLAP | SYSTEM | S0 | S5",
    "action": "start | stop | press | set | fault | restart",
    "position": "S6 | S7 | S8"
  }
}
```

Câmpul **position** este relevant doar când **target = FLAP** și **action = set**.

Sincronizare (bidirecțional)

La conectarea unui nou client, simulatorul trimite starea completă curentă:

```
{
  "type": "sync",
  "timestamp": "...",
  "payload": {
    "system": "Idle | Normal | Fault",
    "flap": "S6 | S7 | S8 | Fault",
    "belts": {
      "M1": "Stopped | Ready | Running",
      "M2": "Stopped | Ready | Running",
      "M3": "Stopped | Running",
      "M4": "Stopped | Running"
    }
  }
}
```

5.2 Comunicare Controller ↔ Aplicație web (WebSocket)

Mesajele sunt mai compacte, orientate spre UI. Controllerul traduce evenimentele interne în mesaje tipizate:

Tipuri de mesaje (Controller → Aplicație)

```
{ "type": "sync", "payload": { "system": "...", "flap": "...", "belts": { ... } } }
{ "type": "belt_state", "payload": { "belt": "M1", "state": "Running" } }
{ "type": "flap_state", "payload": { "position": "S6" } }
{ "type": "system_state", "payload": { "state": "Fault" } }
{ "type": "alarm", "payload": { "active": true } }
{ "type": "alarm", "payload": { "active": false } }
```

Decizie de proiectare — alarmă: Alarma este condusă de server (**active: true/false**), nu de un timer local în browser. Astfel, toți clienții conectați reflectă starea reală a simulatorului, indiferent de latența rețelei.

Comenzi (Aplicație → Controller)

```
{
  "type": "command",
  "payload": {
    "target": "M1 | M2 | M3 | M4 | FLAP | S0 | S5 | SYSTEM",
    "action": "start | stop | press | set | fault | restart",
    "position": "S6 | S7 | S8"
  }
}
```

5.3 Sincronizarea stării la conectare

Controllerul menține în memorie (**ProcessState**) starea curentă a întregului sistem, actualizată la fiecare eveniment primit de la simulator. Când un client WebSocket se conectează, primește imediat un mesaj **sync** cu această stare, fără a fi nevoie să interogheze simulatorul.

6. Implementare — Simulator

6.1 FSM-urile

Fiecare FSM este implementat ca o clasă C# cu:

- o proprietate publică pentru starea curentă (enum)
- metode publice pentru fiecare tranziție posibilă
- un eveniment **StateChanged** care notifică abonații la fiecare tranziție

Implementarea folosește abordarea **enum + switch** în locul pattern-ului State (clase separate per stare), deoarece stările sunt suficient de simple și abordarea mai simplistă reduce boilerplate-ul fără a sacrifica claritatea.

6.2 Evaluatorul de condiții

`ConditionEvaluator` este o clasă statică care implementează tabelul de condiții din secțiunea 2.4 ca o expresie `switch`.

6.3 Orchestratorul procesului

`ProcessOrchestrator` este componenta centrală a simulatorului. Responsabilitățile sale:

1. **Instanțierea și conectarea FSM-urilor** — abonare la evenimentele fiecărei componente
2. **Expunerea comenzilor** — `StartBelt`, `StopBelt`, `StopAllBelts`, `MoveFlap`, `TriggerFault`, `Restart`
3. **Reevaluarea condițiilor** — la fiecare schimbare de stare, evaluează dacă benzile de intrare active trebuie să tranziteze
4. **Actualizarea stării sistemului** — determină când sistemul trece între IDLE și NORMAL
5. **Emiterea evenimentelor** — fiecare tranziție devine un `ProcessEvent` transmis mai departe

Decizie de proiectare: Comenzile în timpul FAULT sunt blocate la nivelul orchestratorului, nu doar în UI. Astfel, comportamentul este consistent indiferent de sursa comenzii (interfață web, consolă de test, API).

6.4 Serverul TCP

`TcpServer` expune procesul prin rețea:

- Acceptă conexiuni multiple simultane
- La conectare trimite imediat un mesaj `sync`
- Retransmite (broadcast) fiecare eveniment al orchestratorului tuturor clienților conectați
- Primește comenzi și le traduce în apeluri pe orchestrator
- Folosește `\n` ca delimitator de mesaje pentru parsing simplu

6.5 Task-ul de perturbații

`PerturbationTask` introduce erori aleatorii de senzori la intervale configurabile (implicit 15–45 secunde). Acționează direct pe `FlapFSM.TriggerFault()` doar când sistemul este în stare NORMAL — perturbațiile nu au efect când procesul este oprit.

7. Implementare — Controller

7.1 Clientul simulatorului (SimulatorClient)

`SimulatorClient` este un `BackgroundService` ASP.NET Core care:

- Se conectează la simulator prin TCP la pornire
- Reconectează automat în caz de pierdere a conexiunii (retry la 3 secunde)
- Menține referința la stream-ul activ pentru trimiterea comenzilor
- Folosește `SemaphoreSlim` pentru acces thread-safe la stream

Parsarea mesajelor gestionează cazul în care un mesaj TCP poate sosi fragmentat în mai multe citiri (`leftover` buffer pentru linii incomplete).

7.2 Starea procesului în memorie (ProcessState)

`ProcessState` menține o copie a stării curente a procesului, actualizată sincron la fiecare eveniment primit. Servește mesajul `sync` instantaneu clienților WebSocket la conectare, fără latență suplimentară.

7.3 Hub-ul WebSocket (WebSocketHub + WebSocketMiddleware)

`WebSocketHub` gestionează lista de clienți WebSocket conectați și expune metode de broadcast și trimitere individuală.

`WebSocketMiddleware` interceptează cererile pe calea `/ws`:

1. Acceptă conexiunea WebSocket
2. Trimite `sync` imediat
3. Pornește o buclă de primire a comenzilor
4. La deconectare, elimină clientul din hub

7.4 Logger și rapoarte

Jurnalizarea se face ca efect secundar al procesării evenimentelor în `SimulatorClient` — fiecare eveniment de tip tranziție FSM este scris în baza de date MSSQL prin Entity Framework Core.

`ReportService` oferă două interogări:

- **GetEventsAsync** — filtrabilă după sursă și interval de timp
- **GetFaultPeriodsAsync** — corelează evenimentele `SYSTEM→Fault` cu evenimentele `SYSTEM→Idle` pentru a calcula duratele perioadelor de întrerupere

7.5 Baza de date

Schema bazei de date este minimalistă — un singur tabel `ProcessEvents`:

Coloană	Tip	Descriere
Id	int (PK)	identificator unic
Source	nvarchar	sursa evenimentului (M1, FLAP, SYSTEM etc.)
From	nvarchar	starea anterioară
To	nvarchar	starea nouă
Timestamp	datetime2	momentul evenimentului (UTC)

Migrațiile sunt gestionate prin EF Core și aplicate automat la pornirea controller-ului.

8. Implementare — Aplicație de monitorizare

8.1 Tehnologii și structură

Aplicația web este implementată în SvelteKit cu TypeScript și Tailwind CSS. Folosește Svelte 5 cu sistemul de reactivitate bazat pe rune (`$state`, `$derived`, `$props`).

8.2 Store-ul de conexiune

`connection.svelte.ts` este nucleul aplicației — un store reactiv Svelte 5 care:

- Gestionează conexiunea WebSocket (conectare, deconectare, reconectare automată la 3s)
- Actualizează starea procesului la fiecare mesaj primit
- Menține un jurnal local de evenimente din sesiunea curentă (ultimele 200)
- Expune metode pentru toate comenzile posibile

8.3 Vizualizarea procesului

Schematica procesului este implementată ca SVG, urmărind topologia din documentul original:

- Benzile sunt reprezentate ca dreptunghiuri cu linii punctate animate
- Animația benzilor (`stroke-dashoffset`) este activă doar când banda este în stare RUNNING
- Clapeta este reprezentată printr-un indicator rotativ care reflectă poziția curentă
- Fiecare componentă are un indicator LED colorat:
 - Gri — STOPPED

- Chihlimbar — READY
- Verde — RUNNING
- Roșu pulsant — FAULT

8.4 Pagini

Pagina Process (/) — vizualizare în timp real:

- Bannerul stării sistemului (IDLE / NORMAL / FAULT + indicator alarmă)
- Schematica SVG a procesului cu animații
- Carduri de stare per bandă cu butoane start/stop
- Panoul de control (selector clapetă, S0, S5, trigger fault, restart)

Pagina Event log (/log) — jurnal evenimente:

- Evenimente live din sesiunea curentă (din store)
- Evenimente istorice din baza de date (prin HTTP)
- Filtrare după sursă

Pagina Reports (/reports) — rapoarte audit:

- Tabel cu perioadele de întrerupere a procesului
- Timestamp-uri de intrare și ieșire din FAULT
- Durata fiecărei întreruperi
- Marcaj vizual pentru întreruperi nerezolvate

8.5 Responsivitate

Interfața este proiectată mobile-first folosind Tailwind CSS. Navigarea este implementată ca tab-uri (top pe desktop, utilizabile pe orice dimensiune de ecran). Layout-ul paginii principale folosește un grid `md:grid-cols-2` care colapsează la o singură coloană pe ecrane mici.

9. Testare

9.1 Teste unitare automate

Testele sunt implementate cu xUnit și acoperă:

Modul	Teste
FlapFSM	tranziții valide, tranziții invalide, evenimente
InputBeltFSM	toate tranzițiile stărilor, idempotență
OutputBeltFSM	pornire/oprire, evenimente
SystemFSM	toate tranzițiile, guard-uri
ConditionEvaluator	toate combinațiile din tabelul de condiții

Modul	Teste
ProcessOrchestrator	scenarii end-to-end (S6/S7/S8, S0, S5, FAULT)
TcpServer	conectare, sincronizare, comenzi prin rețea
PerturbationTask	declanșare în NORMAL, ignorare în IDLE
ProcessState	actualizări, snapshot
ReportService	filtrare evenimente, perioadele de FAULT

10. Cerințe de sistem

- .NET SDK 8.0 sau mai nou
- Node.js 20+ și npm
- Docker (pentru MSSQL pe Linux/macOS)
- `dotnet-ef: dotnet tool install --global dotnet-ef`